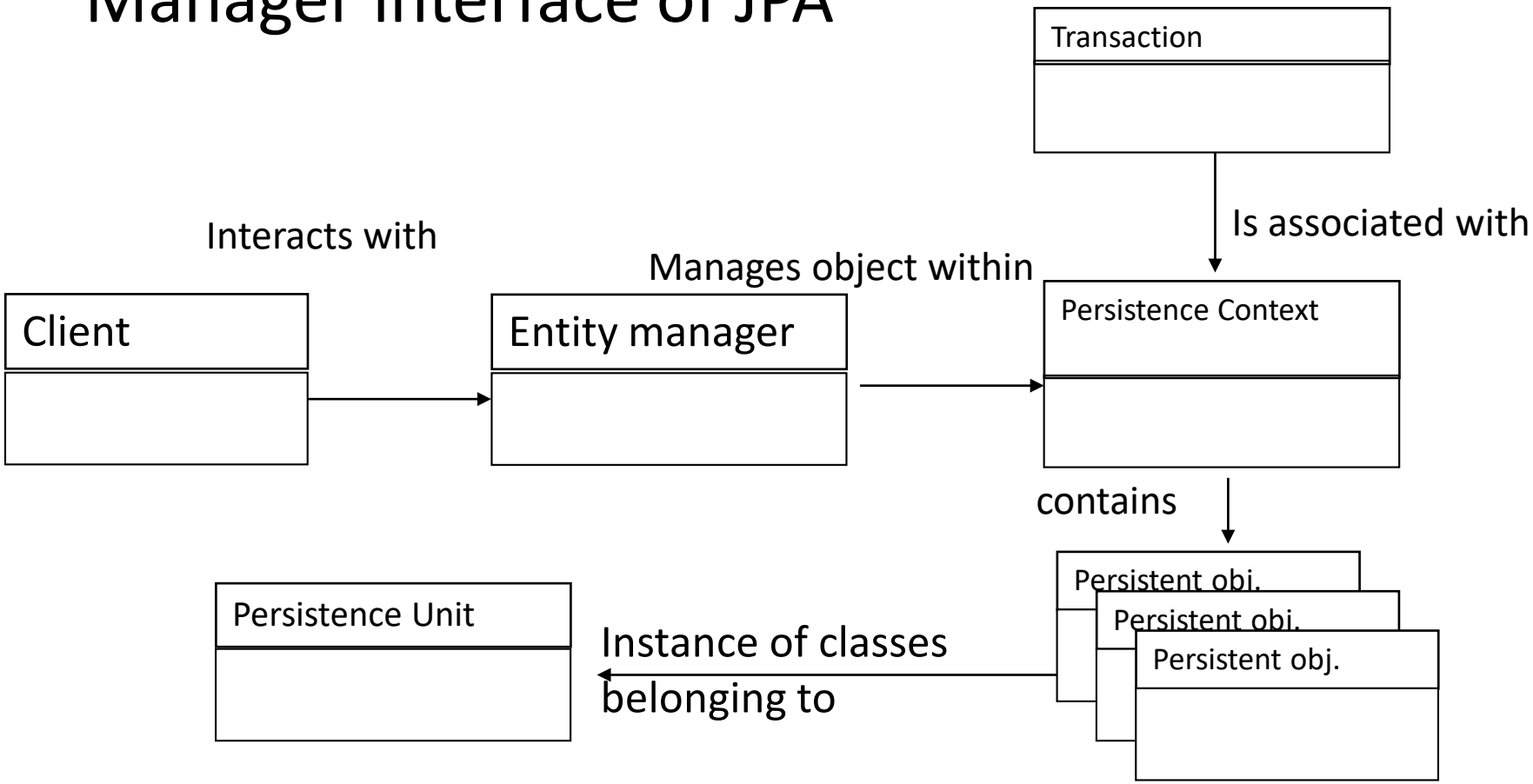
Java Persistence API
The Entity Manager

Databases 2

How to work with entities

- Entities are accessed through the Entity Manager interface of JPA



EntityManager

- Because entity instances are plain Java objects, they **do not become managed until the application invokes an API method to initiate the process**
- The Entity Manager is the central authority for all persistence actions
  - It manages the mapping between a fixed set of entity classes and an underlying data source
  - It provides APIs for creating queries, finding objects, and synchronizing objects and database state

Persistence Context

- It is the **fundamental (and somehow elusive) concept of JPA:** a kind of main memory database that holds the objects in the managed state
- A managed object is **tracked**, i.e., all the modifications to its state are monitored for **automatic alignment to the database**
- Database writes by default occur **asynchronously**, at a time decided by the persistence provider implementation
- For the database writes to happen, the **Persistence Context must hook up to a transaction**, which is the only means to write to the database
- A managed entity has **two lives**: one as a Java object and one as a relational tuple bound to it (ID of the POJO = PK of the tuple)
- Such a binding exists **ONLY inside the persistence context.** When the POJO exits the persistence context the binding breaks: it gets untracked and no longer synchronized to the database
- NOTE: the application **never sees the persistence context**, it interacts ONLY with the Entity Manager

`EntityManager` Interface

Method signature	Description
`public void persist(Object entity);`	Makes an entity instance become part of the persistence context, i.e., managed
`public <T> T find(Class<T> entityClass, Object primaryKey);`	Finds an entity instance by its primary key, the entity becomes managed
`public void remove(Object entity);`	Removes an entity instance from the persistence context and thus from the database
`public void refresh(Object entity);`	Resets the state of entity instance from the content of the database
`public void flush();`	Writes the state of entities to the database as immediately as possible

Creating a new POJO

- Calling the `new` operator **does not** magically interact with some underlying service to create the `Employee` in the database
  - Instances of the `Employee` class remain POJOs until the application asks the `EntityManager` to start managing the entity
- When an entity is first instantiated, it is in the **transient** (or **new**) state since the `EntityManager` **does not know it exists yet**
- Transient entities are not part of the persistence context associated with the Entity Manager

Creating a new POJO

```
Employee emp = new Employee(ID, "John Doe"); //emp unknown to the EM!
```

Making an entity managed

- Entity Manager's `persist()` method makes a transient entity become **managed** **(BIG WARNING: managed <> written to the DB!)**
- A managed entity "lives" in a persistence context and the `EntityManager` makes sure that any change to its state is tracked for being persisted to the database, **sooner or later**
- When the transaction associated with the Entity Manager's persistence context commits a new record is created in the database mapping the entity
- The Entity Manager `flush()` method can be called to ask the Persistence provider to write changes as soon as possible to the database
- The managed entity and the corresponding tuple become associated until the entity exits the managed state: changes to the entity will be reflected to changes to the tuple
- NOTE: calling `persist()` on an already managed entity instance is possible and triggers the cascade process

Persisting an entity

```
Employee emp = new Employee(ID, "John Doe");
em.persist(emp); // emp now is managed
```

Finding an entity

- `EntityManager`'s `find()` method takes as an input the class of the entity that is being sought and the primary key value that identifies the desired entity instance
- When the call completes, the returned object will be **managed**, added to the persistence context
  - If the entity instance is not found, then the `find()` method returns `null`
- The actual amount of data extracted and added to the persistence context depends on the **fetch policy** of attributes and relationships

Finding an entity

```
Employee emp = em.find(Employee.class, ID); //emp is managed
```

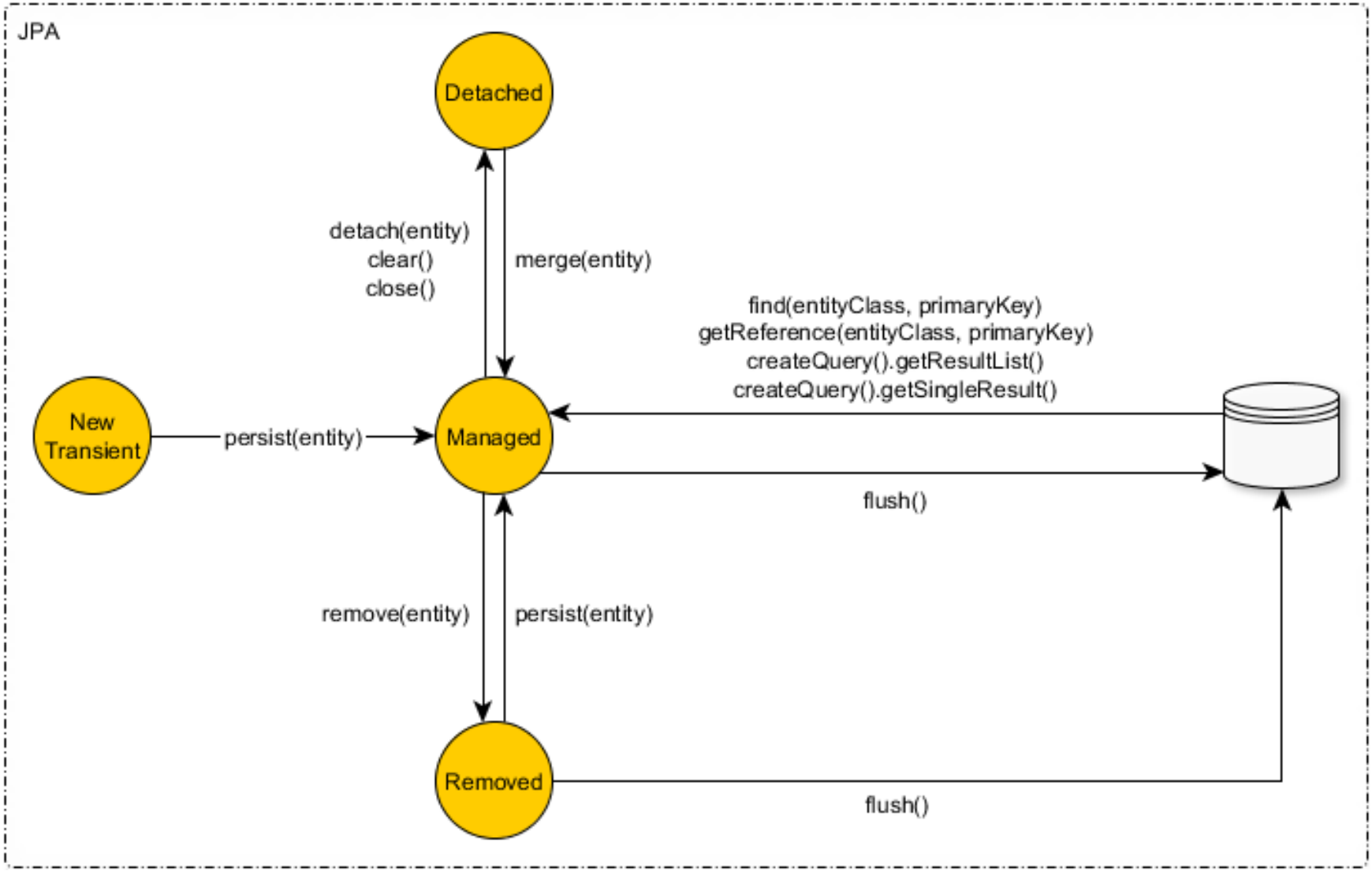
Removing an entity

- Entity Manager's `remove()` method breaks the association between the entity and the persistence context
- When the transaction associated with the Entity Manager's persistence context commits or the Entity Manager `flush()` method is called, the tuple associated with the entity is scheduled for deletion from the database
- The entity (Java object) still exists but its changes are no longer tracked for being (later) synchronized to the database

Removing an entity

```
em.remove(emp); // emp removed, no longer managed
```

EM operations & entity state transitions



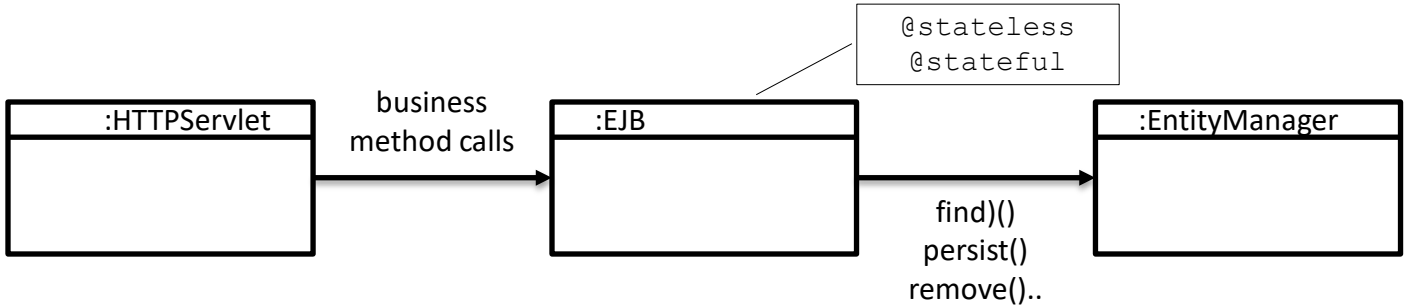
Credits https://vladmihalcea.com/a-beginners-guide-to-jpa-hibernate-entity-state-transitions/

Entity lifecycle legenda

- NEW: Unknown to the Entity Manager, no persistent identity, no tuple associated
- MANAGED: Associated with persistence context, changes to objects automatically synch to database (**NOT VICE VERSA!**)
- DETACHED: Has an identity potentially associated with a database tuple but changes are NOT automatically propagated to database
- REMOVED: Scheduled for removal from the database
- DELETED: erased from the database

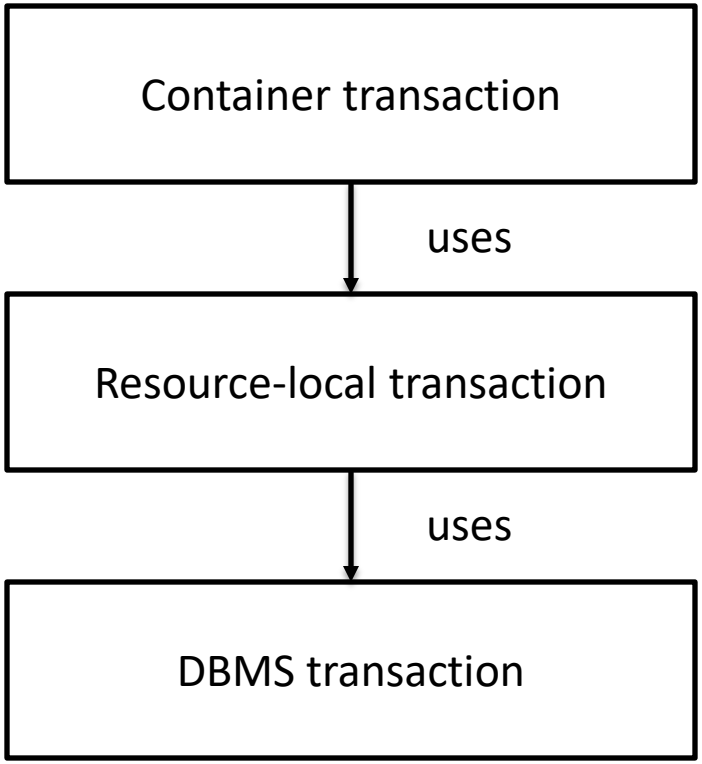
JPA application architectures

- JEE
  - The client exploits the services of a container (EJB or CDI) to connect to the Entity Manager
    - The client (e.g., a Web servlet) interacts with a business object (e.g., an EJB or a CDI component)
    - The business component interacts with the Entity Manager
  - The container provides the support to make JPA entity method calls **transactional** through the automatic creation of transactions



Transaction management in JPA

- Database application development requires understanding the transactional properties of code
- How the Entity Manager participates in transactions
- When a transaction starts/ends
- Even if transactions are managed transparently by the container, understanding how they work is necessary to predict application behaviour
- Transactions exist at three levels
  - DBMS transactions
  - Resource-local transactions
  - Container transaction



Transaction abstraction levels

- DBMS
  - They live inside the DBMS are demarcated by SQL commands and managed by the DBMS
- Resource-local
  - They are created and demarcated by means of JDBC commands issued through the JDBC connection interface. They are mapped by the JDBC driver to DBMS transactions. **They must be managed by the application**
- Container (we will use this level)
  - They are defined through the JTA interface and mapped by the JTA Transaction Manager and EJB container to JDBC transactions. **They can be managed by the application or by the container (EJB, servlet or CDI)**

CM Entity Manager

- Container Managed

```
@Stateless // this is a business object (EJB)
public class myEJBService {
  @PersistenceContext(unitName = "MyPersistenceUnit")
  private EntityManager em;
```

- The container injects the EM instance into the business object
- The container creates and destroys instances of the Entity Manager transparently to the application
- This is the EM type **used by default**, if no other specification is added
- The container **provides the transaction** needed for saving the modifications made to the entities of the Persistence Context associated with the Entity Manager into the database

CM transaction: example

- (Web) client code (e.g., in a servlet)

```
@EJB(name = "it.polimi.db2.mission.services/MissionService")
private MissionService mService; // client obtains a business object
. . .
Mission mission = mService.closeMission(missionId, userId);
```

- Business component code MissionService EJB

```
// transaction started here by the container
public void closeMission(int missionId, int reporterId) throws
                        BadMissionReporter, BadMissionForClosing {

 // persistence context created and associated with transaction
 Mission mission = em.find(Mission.class, missionId);
 if (mission.getReporter().getId() != reporterId) {
                         throw new BadMissionReporter(" . . .");
 }
 if (mission.getStatus() != MissionStatus.REPORTED) {
        throw new BadMissionForClosing(" . . . ");
 }
 mission.setStatus(MissionStatus.CLOSED);

} // transaction committed by container
  // no explicit transaction demarcation code needed!
```

Transaction and method calls

- When a client (e.g., a Web Servlet) calls a method of a business object that exploits a (default) CM EM for persistence, the (EJB) container provides (creates or reuses) a transaction for saving the modifications to the database
- If the called method in turns calls another method of the same or of a different business object, the same transaction is reused
- This is the most common and default behavior, but the business objects methods can be annotated to specify a different way to use the transactions provided by the container

Transactional behaviour of methods

- When **Container Managed** transactions are used the container wraps the method calls of managed components and execute them in a transaction
- A method can be annotated to obtain the desired transactional behaviour with `@TransactionAttribute(TransactionAttributeType.XXX)`
- Where XXX can be:
  - MANDATORY: a transaction is expected to have already been started and be active when the method is called. If no transaction is active, an exception is thrown
  - **REQUIRED: the default, if no transaction is active one is started. If one is active (e.g., created by the caller) this is used**
  - REQUIRES_NEW: the method always needs to be in its own transaction. Any active transaction is suspended
  - SUPPORTS: the method does not access transactional resources, but tolerates running inside one if it exists
  - NOT_SUPPORTED: the method will cause the container to suspend the current transaction if one is active when the method is called
  - NEVER: the method will cause the container to throw an exception if a transaction is active when the method is called.

Benefits of JPA: summary

- No need to write SQL code
- Application code ignores table names
  - Mapping is expressed via annotations with defaults
- No need to create/destroy the connection
- No need to create and terminate the transaction (begin, commit, rollback managed by the container)
- Business objects methods automatically made transactional (“all or nothing” semantics at the method call level and not at the SQL statement execution level)
- Default transactional behaviour, with annotations to specify different transaction management policies